CocktailAI: An Evolutionary Recipe Generation Engine on Amazon Web Services (AWS)

Module Assignment for

CS5024 - Theory and Practice of Advanced AI Ecosystems

Student Name : **Jai Parakh**

Student ID : **25165933**

Revision Timestamp: 24/04/2026 19:34:37

Module Director: Patrick Denny
Teaching Assistant: Ankita Saini

Abstract

CocktailAI is an AI powered cocktail recipe generator that uses Genetic Algorithm (GA) driven by 9 LogisticRegression taste classifiers that power GA's fitness function. It generates a recipe based on the user specified taste profiles and if any mandatory ingredients. The motivation for this project goes back in the month of December 2025, when I wanted to create my own cocktails, but GenAI was failing at truly new ideas, so I was curious if the ability of Evolutionary Algorithms driven by simple ML models, can be utilized here to search in a large search space. Now the main idea is to deploy this on AWS, while minimizing the cost but also maximising the scalability at the same time. This report will list all the services used to deploy the CocktailAI, focusing on the principles from AWS Well Architected Framework Design [1]. The ecosystem uses SageMaker, Lambda, SQS, DynamoDB, CloudWatch, X-Ray and IAM roles on Amazon Web Services. All the mentioned services are created via dashboard manually except SageMaker endpoint which was deployed via code in the notebook.

Contents

Introduction

From a functional perspective, the user inputs their desired taste profile, and optional ingredient (s) if any like gin, vodka, etc. Now, it's the AI's job to figure out what ingredients with their respective quantities should be mixed, such that the generated cocktail has the desired taste profile.

The solution consists of two main components, the first is a logistic regression models for taste classification, one per flavour category (bittersweet, citrus, creamy, floral, fruity, herbal, spicy, savoury, sweet) totalling to 9 such models with a custom threshold. These taste predictors are used for fitness calculation for the second component which is a Genetic Algorithm (GA) that evolves a population 1000 individuals over 150 generations, maximising the fitness of the individual as per the taste requirements while penalising unnecessary ingredient complexity.

One key engineering challenge here was the fitness evaluation for a 1000 individuals per generation, which if deployed as a single SageMaker endpoint would be called 150 * 1000 times for the entire GA loop. This violates two pillars of the AWS Architected Framework Design Principles [1], namely Performance Efficiency and Cost Optimization. So, the core optimization introduced here was batch evaluation. Essentially, the entire population is serialised as a numpy array and then sent to the SageMaker endpoint, meaning only one call per generation, this reduced the request from 150k to just 150 for entire GA loop. This not only reduced the cost but also reduced the time for the GA loop to run, hence minimizing the cost of both Lambda and SageMaker.

To summarize the user adds the required ingredients and taste profile, which is then sent to CocktailAI, that creates the recipe that fits the user taste specifications, all in a minute approximately!

AI Ecosystem Architecture Used

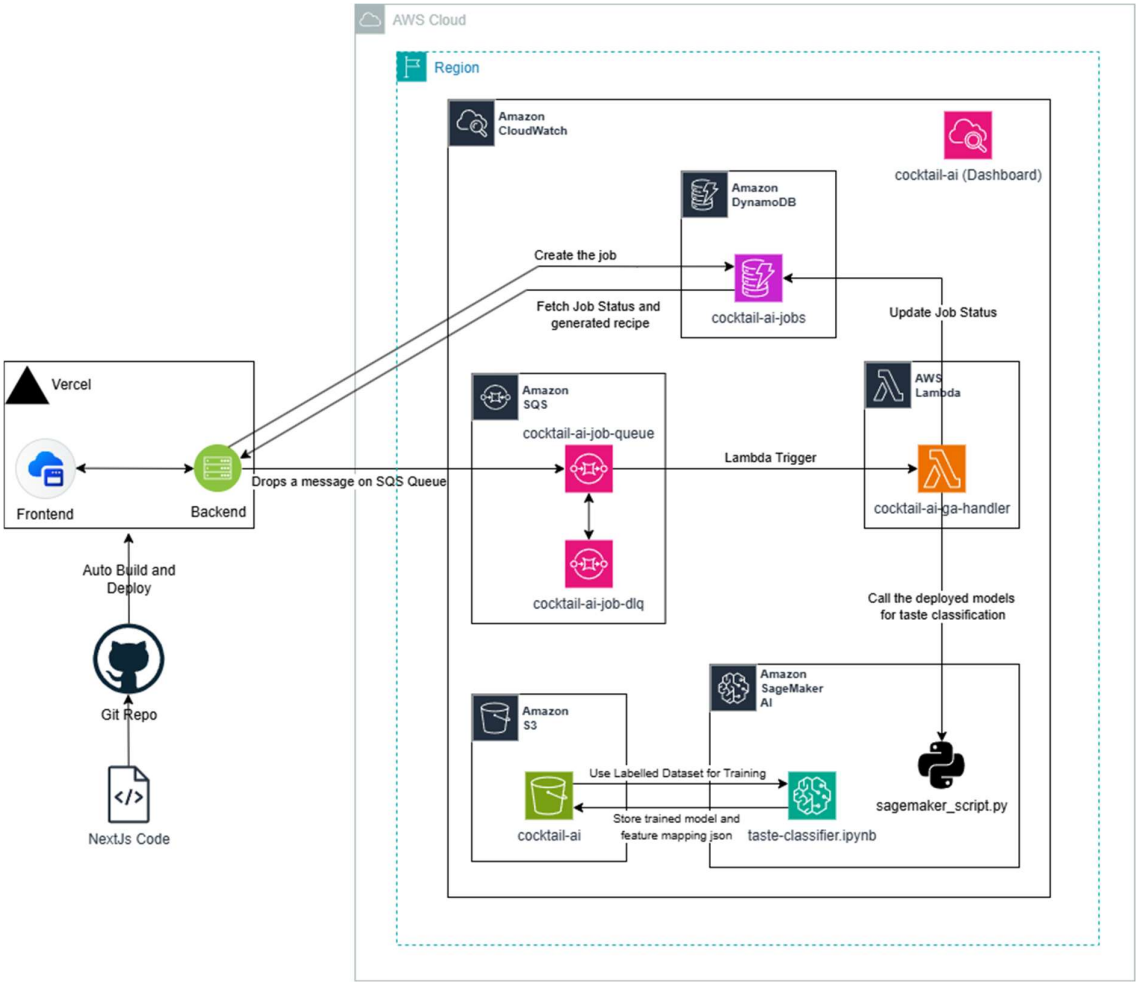

Figure 1: Architecture used to create AI/ML ecosystem for CocktailAI

Figure 1 shows the complete AWS AI Ecosystem used for deploying CocktailAI. The architecture follows an asynchronous job pattern and can be broken down into five layers, frontend and backend application layer, job queuing, ML inference, database and observability.

Frontend and Backend Layer

For frontend I have used NextJs with react and TailwindUI, which is deployed on Vercel, which behind the scenes uses AWS heavily! The backend exposes two routes, first a POST /api/cocktail-ai/jobs that creates a job in Amazon DynamoDB with a status of QUEUED, and then drops a message on SQS queue that triggers the lambda. The second route is GET endpoint, /api/cocktail-ai/jobs/:job_id tht polls DynamoDB for job status once every 1.5 seconds. It returns the entire record each time, so as soon as the status changes to COMPLETE, the recipe is displayed on the UI and the polling stops.

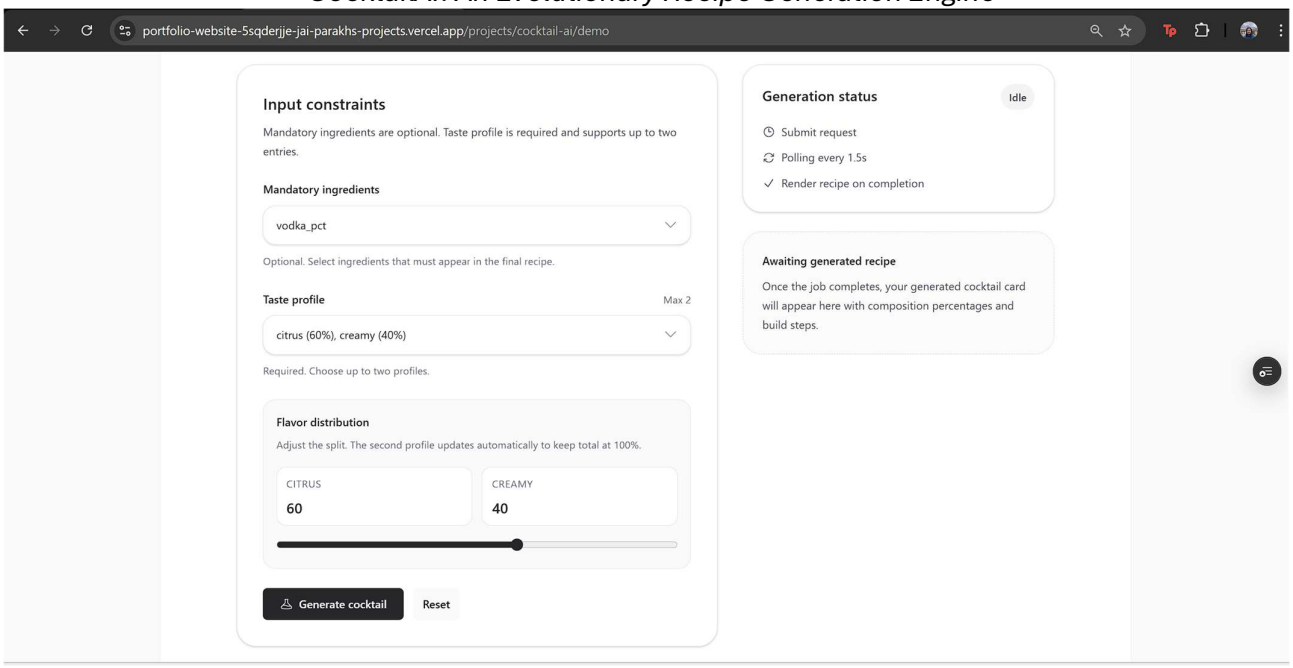

Figure 2: UI deployed on Vercel

Now the three main reasons for using Vercel instead of AWS EC2 for just this layer were, first since NextJs is a server side rendering engine, it already has support for backend APIs, this puts all the web related code in a single repository, making it easier to manage and scale. Second, reason is the cost of the solution, so a shared t4g.small instance with metrics enabled costs around 10.57 USD as per AWS pricing calculator [3] monthly, as compared to 0 USD for a Vercel hosted solution. Given that we just have to poll for the job status and drop an SQS queue event, an ec2 instance running a flask server felt unnecessary, not to mention the cost of an additional VPC and the developer cost, so the Total Cost of Ownership [4] in Vercel is far less compared to Amazon EC2 instance. And thirdly, like Prof. Patrick always says, “Don’t be married to your tools”.

Job Queuing Layer

AWS SQS is the trigger for a lambda. I decided to use this instead of SNS mainly because of two reasons, first as per the AWS decision guide on SNS vs SQS vs EventBridge [5], SNS is mainly used for subscriber based notification service which can be used for Lambda but the guide specifically says that SQS should be used for decoupling microservices, buffering requests and processing tasks asynchronously [5]. The asynchronous behaviour is very important for us since the entire GA loop takes about 45 seconds to run, which would lead to the timeout of HTTP request. So, I decided to trigger lambda with SQS instead of direct call or SNS queue.

The main queue is cocktail-ai-job-queue, which is configured with a visibility timeout of 16 minutes, which is a minute longer than lambda allowed runtime of 15 minutes. Following the reliability pillar of AWS’s design principles [1], the main queue has a supporting dead-letter-queue with name cocktail-ai-job-dlq, which receives the message after 3 failed attempts.

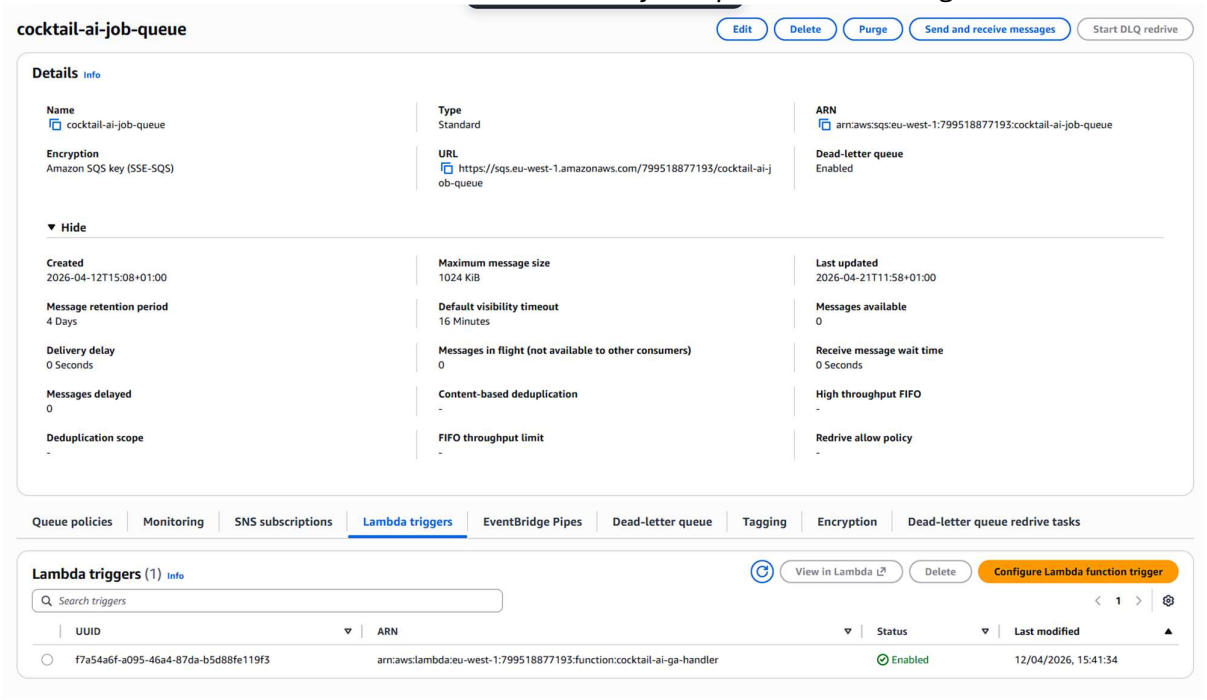

Figure 3: cocktail-ai-job-queue configuration

Compute and ML Inference Layer

Lambda

The Lambda function cocktail-ai-ga-handler is triggered by the above mentioned SQS queue as shown in Figure 2 above. Upon receiving a message, it first updates the job status in the AWS Dynamo DB table, cocktail-ai-jobs, after that it runs the full GA loop, which calls the AWS SageMaker endpoint once per generation, thanks to the above mentioned optimization. The Lambda is configured with a memory of 3GB and a default timeout of 15 minutes. It is also configured with two environment variables, namely DynamoDB table name and sagemaker endpoint. Once the process is completed the job status is updated directly from AWS Lambda, so Frontend knows when to display the recipe.

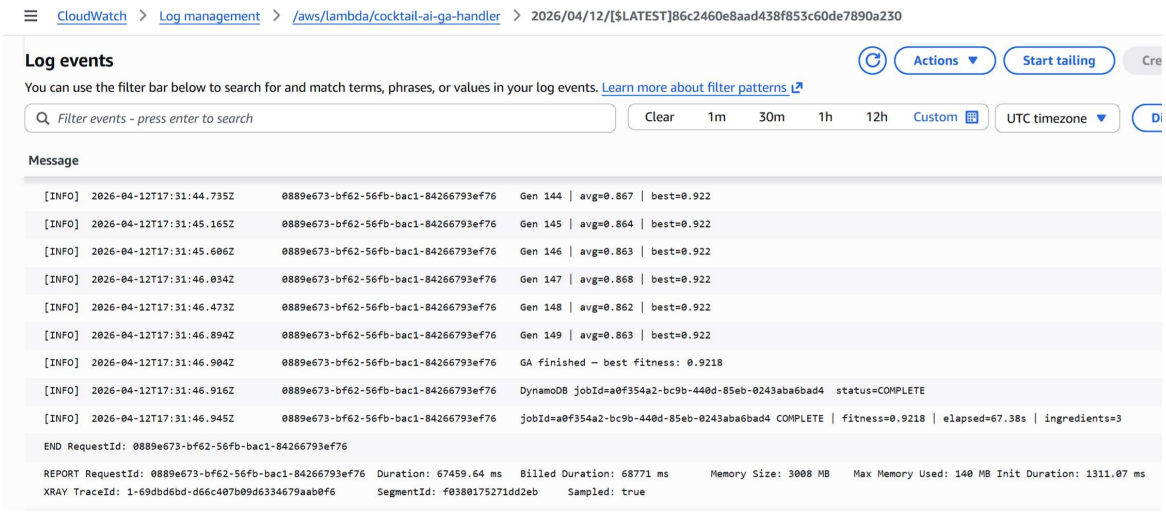

Figure 4: Lambda Logs for a Generated Recipe

SageMaker

The endpoint taste-predictor is a SageMaker SKLearn inference endpoint that runs on ml.m5.large instance. It exposes a custom input_fn/predict_fn/output_fn inference that accepts a 2D numpy array of GA population and returns a JSON dictionary of 9 flavour categories to a vector of probabilities, one per individual. Building the custom inference was difficult, I went through a lot of articles but the one that helped me the most was a Github Link from amazon-sagemaker-examples [6]. So, I created a separate file sagemaker_script.py which included these functions.

The only reason for building the custom inference was the optimization which I needed for my GA's fitness evaluation, it was a hassle, but the cost saving was worth it, following the AWS's cost optimization pillar [1] and the speed boost because of this which follows the performance efficiency pillar [1]

A SageMaker Notebook was also used to train our 9 models. The instance type for the notebook is ml.t3.medium, with a volume size of 5GB EBS.

Database and S3

The motivation for using AWS DynamoDB was mainly the good design principles for asynchronous jobs, since it gives a smoother user experience. Before coming here, I used to work with an AI based marketing company called Blkbox.ai [7]. They had a lot of processing jobs that used EC2 and lambda for report generation to running AI models, so to keep a track of the status, they setup a postgreSQL table that maintained the status of each job with the corresponding output and errors if any.

The main reason for choosing DynamoDB over RDS was the NoSQL structure of the data, and the data is to be only queried by the partitioned key [8] which is jobId.

The Model artifacts after training were stored in an S3 Bucket named cocktail-ai. The bucket also stores the feature columns for our logistic regression models and the dataset used to train the models.

Figure 5: A sample record in cocktail-ai-jobs table

Observability

For Observability a custom CloudWatch Dashboard named cocktail-ai was created which receives custom metrics published by Lambda function after every generation, like BestFitness, AvgFitness, ExecutionTime and FinalFitness score. It also includes metrics from services like Lambda, DynamoDB, SQS, S3 and SageMaker (SageMaker Endpoint metrics). AWS Xray is enabled on the lambda function with patch_all() function call, that further wraps the SageMaker endpoint call.

Lastly AWS Budget was also configured for a monthly limit of 1 USD.

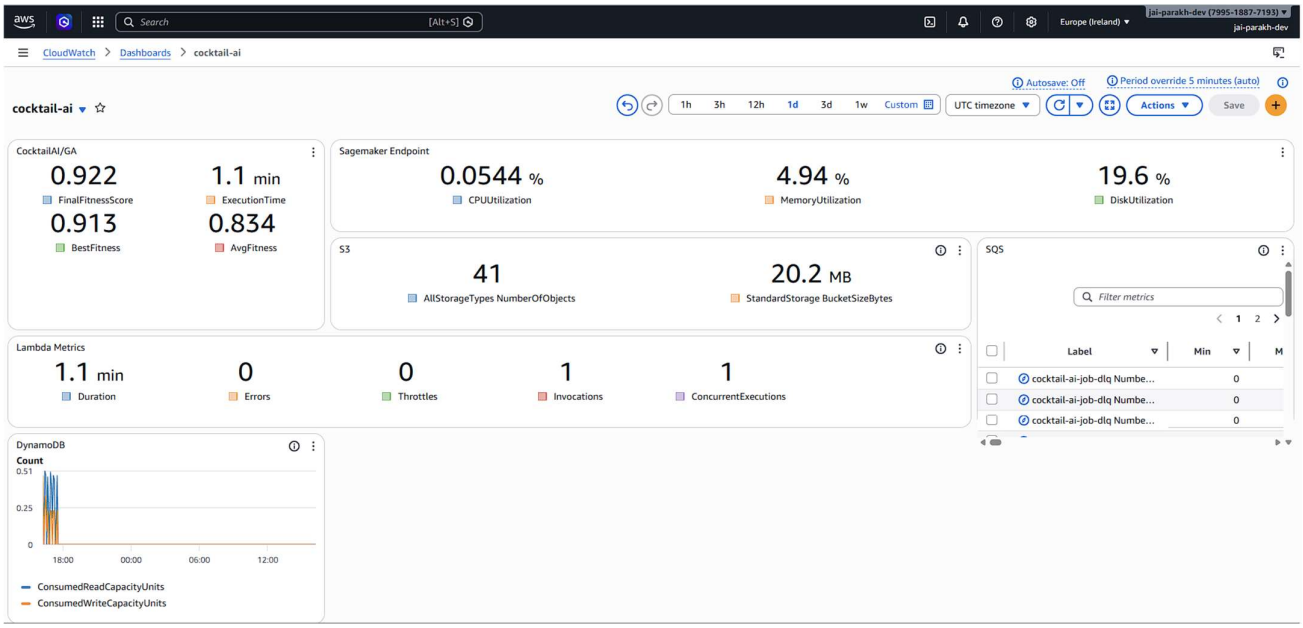

Figure 6: cocktail-ai AWS CloudWatch Dashboard

Model Description

Dataset

The dataset for training the taste predictors was scraped from Difford's Guide website [2]. I was able to scrape a total of 4500 recipes but only 2700 had 10 labelled taste tags. The raw data had a lot of issues like, same ingredients with different names (fresh lemon, lemon), volumes were expressed in inconsistent units (ml, ounces, dash, parts), and the lack of taste profile tag. Substantial feature Engineering and Data processing was applied to make the data usable.

Nutty flavour was dropped first since it only had 8 recipes, this reduced the training data size to 2692 records. The original ingredient count was 683 but 443 of them had less than 10 occurrences, so they were grouped according to their category and taste, for example, fruits, malt whisky, etc. The final ingredient or feature count for our taste predictor models was 242. Sadly, most of this was manual processing.

Taste Classifiers

Nine independent logistic regression classifiers with L1 Regularization were trained, one for each flavour profile (bittersweet, citrus, creamy, floral, fruity, herbal, savoury, spicy and sweet). Each

model outputs a probability score for that flavour. The taste is decided by a custom threshold, different for each model, that was set after a lot of experiments. This was done because the dataset was heavily imbalanced. This score along with the custom threshold was directly used in fitness calculation of an individual.

A stratified 80/20 train-test split was performed, with rare flavour combinations grouped into a single bucket to avoid stratification failures.

The 9 classifiers are packaged into a single CocktailTastePredictor wrapper class and saved as a single joblib artifact. The inference script exposes a predict_probaba method that accepts the optimized numpy 2D array and returns a dictionary mapping each flavour to a probability vector. This is responsible for the optimization mentioned above in the introduction.

Flavour	AUC	F1	Threshold
Creamy	0.995	0.894	0.60
Floral	0.968	0.578	0.90
Bittersweet	0.956	0.800	0.50
Sweet	0.946	0.556	0.80
Spicy	0.944	0.580	0.75
Citrus	0.893	0.772	0.50
Savoury	0.884	0.457	0.80
Fruity	0.881	0.674	0.60
Herbal	0.821	0.516	0.55

Table 1: Per Flavour Classifier Performance and thresholds

Genetic Algorithm

The Genetic Algorithm (GA) was implemented using DEAP framework, which was covered in Evolutionary Algorithms module in the previous semester. Each individual in the population a vector of 242 floating point values representing ingredient percentages. The initial population is set to 1000, with a crossover probability of 0.7 that handles the ingredients required and a mutation probability of 0.3 changes the quantity of an ingredient. The fitness function weights flavour alignment at 80% and recipe simplicity (rewarding fewer ingredients) at 20%, with a penalty for violating ingredient constraints, their quantity and total percentage bound, meaning the total quantity should not go over 1. Gaussian Mutation with tournament selection of size 3 was used for the GA loop that runs for 150 generations, with elitism enabled, so that we don't loose the best candidate solution in the evolution process.

Result

Figure 7 shows the aggregated metrics of all the runs including the test runs, which had some errors hence the error number. As you can see the average final fitness score is 95.2% with an average fitness of 90.6%, demonstrating a strong convergence. These numbers make sense as shown in Figure 8, where the user inputs vodka with a taste profile of 60% sweet and 40% creamy and the AI generates a recipe with three ingredients namely, Lemon Juice, Whipping cream and Vodka.

The SageMaker Endpoint panel shows negligible resource consumption, confirming the above mentioned fitness evaluation optimization works as expected.

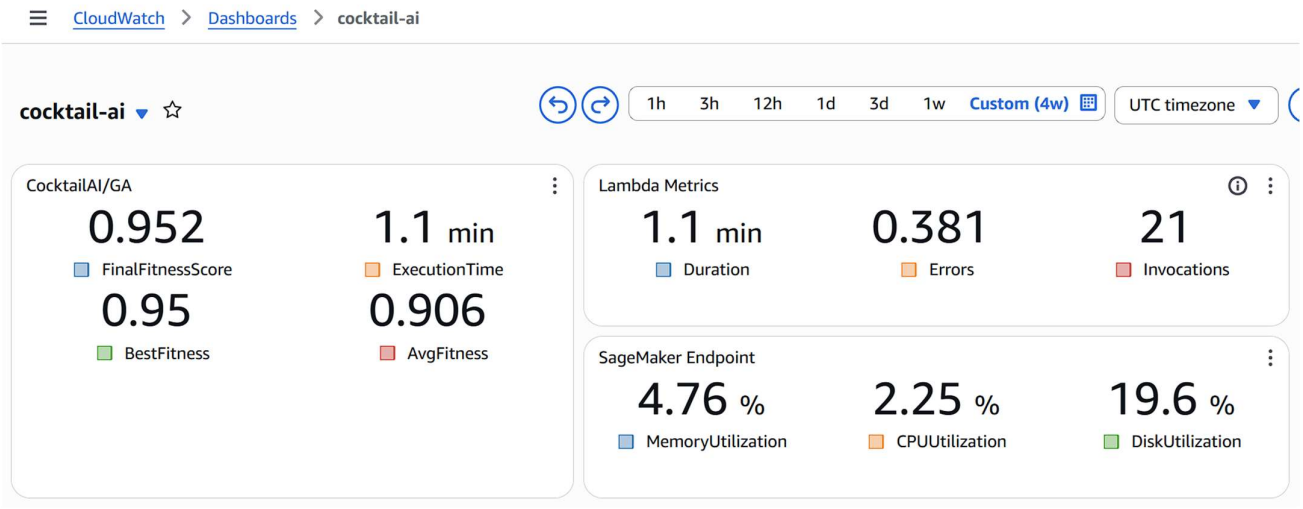

Figure 7: CloudWatch Average Metrics for Results

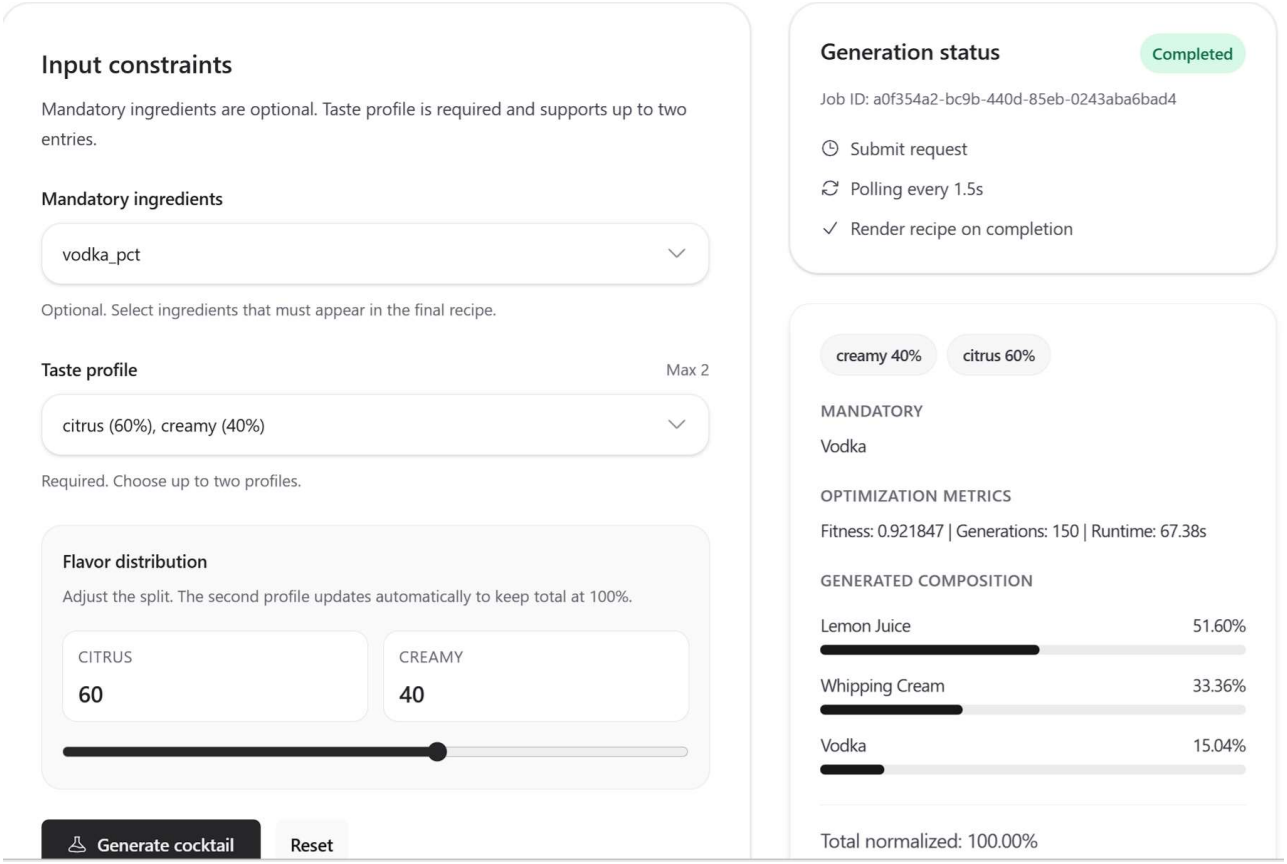

Figure 8: Cocktail AI output on UI

Scalability Considerations

This section examines how the CocktailAI's currently deployed ecosystem would need to evolve across each of the five AWS Well Architected Framework pillars [1]

Operational Excellence

The current architecture has no automated deployment pipeline. All the services were created manually through the AWS cloud console. As mentioned in the AWS CloudFormation [9] document and in the Operation Excellence Pillar of Class Content [1], can be used to setup version control and automatic deployments. Using CloudFormation, a template can be created that describes all the resources and CloudFormation can take care of provisioning and configure those resources. [9]

Security

The SageMaker Endpoint and DynamoDB are currently accessible from Lambda and Vercel Server over the public AWS network. For now, since we are not storing any user specific information it is fine, but tomorrow if we decide to store this information, SageMake Endpoint, DynamoDB, and Lambda must be placed in private subnet with VPC, as taught in Week 4 lectures [10].

Reliability

The SQS Dead Letter Queue already has a basic retry mechanism and since DynamoDB records serve no purpose after generating the recipe a TTL is also setup. But the SageMaker Endpoint is a single point of failure. If it becomes unavailable, all lambda invocations would fail. This can be fixed by implementing Autoscaling and Monitoring [11]. To achieve this application auto scaling can be added. Another thing that can be done is, since we have simple models, SageMaker can be eliminated entirely and a single flask server can be deployed on Amazon elastic EC2, with support for auto scaling as taught in Week 8 [11].

Performance Efficiency

The batch inference optimization mentioned earlier, that computes the fitness of the entire generation in a single endpoint call, is already quite efficient. The GA configuration variables like crossover and mutation rates, selection size and method can further be experimented to get better results. Although logistic regression outperformed other models like XGBoost and RandomForest, many other simpler models can be experimented with which might produce better metrics.

Cost Optimization

The architecture makes several cost optimization decisions. For example, S3 uses standard storage with no replication, DynamoDB is configured with TTL, the SageMaker endpoint invocation is the biggest cost saver of all.

The highest cost is incurred by SageMaker endpoints that run continuously on an ml.m5.large instance regardless of traffic. Since we have simple 9 Logistic Regression Models, an alternate worth considering is to upload them on s3 and load them every time Lambda is invoked. This would reduce the cost significantly, but it violates the Operational Excellence and Security pillars [1]. From Operational Excellence perspective, it removes the separation between model training and usage and requires redeployment of lambda rather than SageMaker redeployment. Now, from security perspective, we will need an s3 bucket with least security privileged IAM role. This exact problem is solved by SageMaker in the current architecture, but it came at the cost of SageMaker endpoint invocation.

References

[1] Patrick Denny. (2026). Cloud Architecture - *Week 6 Material*. Retrieved from CS5024 - Theory and Practice of Advanced AI Ecosystems: https://learn.ul.ie/d2l/le/lessons/73419/units/1185775

[2] https://www.diffordsguide.com/

[3] https://calculator.aws/#/

[4] Patrick Denny. (2026). Cloud Overview, Cloud Economics and Billing - *Week 2 Material*. Retrieved from CS5024 - Theory and Practice of Advanced AI Ecosystems: https://learn.ul.ie/d2l/le/lessons/73419/units/1185771

[5] https://docs.aws.amazon.com/decision-guides/latest/sns-or-sqs-or-eventbridge/sns-or-sqs-or-eventbridge.html

[6] https://github.com/aws/amazon-sagemaker-examples/blob/main/sagemaker-script-mode/sklearn/train.py

[7] https://blkbox.ai/

[8] Patrick Denny. (2026). Storage and Databases - *Week 5 Material*. Retrieved from CS5024 - Theory and Practice of Advanced AI Ecosystems: https://learn.ul.ie/d2l/le/lessons/73419/units/1185774

[9] https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html

[10] Patrick Denny. (2026). Networking, Content Delivery and Compute - *Week 4 Material*. Retrieved from CS5024 - Theory and Practice of Advanced AI Ecosystems: https://learn.ul.ie/d2l/le/lessons/73419/units/1185773

[11] Patrick Denny. (2026). Automatic Scaling and Monitoring - *Week 8 Material*. Retrieved from CS5024 - Theory and Practice of Advanced AI Ecosystems: https://learn.ul.ie/d2l/le/lessons/73419/units/1185776